

A Course Based Project Report on

E - COMMERCE DATA PIPELINE & ANALYTICS SYSTEM

submitted to the
Department of Information Technology
in partial fulfilment of the requirements for the completion of course
DATA ENGINEERING AND MACHINE LEARNING LABORATORY
(22PC1AM202)

BACHELOR OF TECHNOLOGY IN INFORMATION TECHNOLOGY

Submitted by

G. BHAVANI GOUD	23071A1288
J. S. SHRADHA	23071A1292
K. SREE DHARANI KARTHIKA	23071A1295
A. BHAVANI	24075A1208

Under the guidance of

Dr V SARITHA

Assistant Professor, Department of IT, VNRVJIET



DEPARTMENT OF INFORMATION TECHNOLOGY

**VALLURUPALLI NAGESWARA RAO VIGNANA JYOTHI
INSTITUTE OF ENGINEERING & TECHNOLOGY**

An Autonomous Institute, NAAC Accredited with 'A++' Grade, NBA
Vignana Jyothi Nagar, Pragathi Nagar, Nizampet (S.O), Hyderabad – 500 090,
TS, India

MAY 2026

**VALLURUPALLI NAGESWARA RAO VIGNANA JYOTHI
INSTITUTE OF ENGINEERING AND TECHNOLOGY**

An Autonomous Institute, NAAC Accredited with 'A++' Grade,

Vignana Jyothi Nagar, Pragathi Nagar, Nizampet(SO), Hyderabad-500090, TS, India

DEPARTMENT OF INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project report entitled “**E - Commerce Data Pipeline & Analytics System**” is a Bonafide work done under our supervision and is being submitted by **Ms. G.BHAVANI GOUD (23071A1288), Ms. J.S. SHRADHA (23071A1292), Ms. K. SREE DHARANI KARTHIKA (23071A1295), Ms. A. BHAVANI (24075A1208)** in partial fulfilment for the award of the degree of **Bachelor of Technology** in Information Technology, of the VNRVJIET, Hyderabad during the academic year 2025-2026.

Dr. N MANGATHYARU

Professor & HOD, IT

Dr. V SARITHA

Assistant Professor, IT

Course based Projects Reviewer

**VALLURUPALLI NAGESWARA RAO VIGNANA JYOTHI INSTITUTE OF
ENGINEERING AND TECHNOLOGY**

An Autonomous Institute, NAAC Accredited with 'A++' Grade,
Vignana Jyothi Nagar, Pragathi Nagar, Nizampet(SO), Hyderabad-500090, TS, India

DEPARTMENT OF INFORMATION TECHNOLOGY



DECLARATION

We declare that the course based project work entitled “**E - Commerce Data Pipeline & Analytics System**” submitted in the Department of Information Technology, Vallurupalli Nageswara Rao Vignana Jyothi Institute of Engineering and Technology, Hyderabad, in partial fulfilment of the requirement for the award of the degree of **Bachelor of Technology in Information Technology** is a bonafide record of our own work carried out under the supervision of **Dr V. Saritha Assistant Professor, Department of IT, VNRVJIET**. Also, we declare that the matter embodied in this thesis has not been submitted by us in full or in any part thereof for the award of any degree/diploma of any other institution or university previously.

G. BHAVANI GOUD	23071A1288
J. S. SHRADHA	23071A1292
K. SREE DHARANI KARTHIKA	23071A1295
A. BHAVANI	24075A1208

ACKNOWLEDGEMENT

We express our deep sense of gratitude to our beloved President, Sri. D. Suresh Babu, VNR Vignana Jyothi Institute of Engineering & Technology for the valuable guidance and for permitting us to carry out this project.

With immense pleasure, we record our deep sense of gratitude to our beloved Principal, **Dr. B. Chennakesava Rao**, for permitting us to carry out this project.

We express our deep sense of gratitude to our beloved Professor **Dr V. Saritha**, Assistant Professor, Department of Information Technology, VNR Vignana Jyothi Institute of Engineering & Technology, Hyderabad-500090 for the valuable guidance and suggestions, keen interest and through encouragement extended throughout the period of project work.

We take immense pleasure to express our deep sense of gratitude to our beloved Guide, Professor **Dr V. Saritha** ,Assistant Professor, Department of Information , VNR Vignana Jyothi Institute of Engineering & Technology, Hyderabad, for his/her valuable suggestions and rare insights, for constant source of encouragement and inspiration throughout my project work.

We express our thanks to all those who contributed for the successful completion of our project work.

G. BHAVANI GOUD	23071A1288
J. S. SHRADHA	23071A1292
K. SREE DHARANI KARTHIKA	23071A1295
A. BHAVANI	24075A1208

TABLE OF CONTENTS

S No	Contents	Page No
1.	ABSTRACT	1
2.	INTRODUCTION	2-3
3.	SOURCE CODE	4-10
4.	OUTPUT	11-12
5.	CONCLUSION	13
6.	REFERENCES	14

ABSTRACT

This project presents the design and implementation of an end-to-end E-commerce Data Pipeline and Analytics System that transforms raw transactional data into meaningful insights. The system begins with the collection of datasets containing information on customers, products, and orders. Data preprocessing techniques such as handling missing values, removing duplicates, and selecting relevant features are applied to ensure data quality and consistency.

Further, feature engineering is performed to derive valuable metrics like total customer spend and order frequency, enabling deeper behavioural analysis. Statistical methods including mean, variance, and correlation are used to uncover patterns and relationships within the data, along with distribution analysis to identify sales trends.

The processed data is stored in a NoSQL database (MongoDB), allowing flexible and scalable data management. Aggregation queries are used to extract insights such as top-performing customers and product trends. Additionally, big data concepts are introduced by simulating large datasets and explaining the Hadoop ecosystem workflow. This project demonstrates practical applications of data engineering, analytics, and database management in an e-commerce context.

1. INTRODUCTION

1.1 PROBLEM DEFINITION

The rapid growth of e-commerce platforms has led to the generation of large volumes of raw and unstructured data, making it difficult to extract meaningful insights for decision-making. Businesses often face challenges in handling missing or inconsistent data, identifying key customer behaviours, and analysing sales trends efficiently. Without a structured data pipeline, valuable information remains underutilized, affecting strategic planning and performance optimization. Therefore, there is a need to design a system that can efficiently preprocess, organize, store, and analyse e-commerce data. This project aims to address these challenges by building a scalable data pipeline and analytics system to transform raw data into actionable insights.

1.2 OBJECTIVE

The objective of this project is to design and implement an efficient e-commerce data pipeline that preprocesses raw data, performs feature engineering and statistical analysis, and stores it in MongoDB. The system aims to generate meaningful insights, such as customer behaviour and sales trends, to support better decision-making.

Key Objectives:

Automated Data Collection & Integration: The system will gather and integrate raw e-commerce datasets (orders, customers, products) from multiple sources, ensuring seamless data flow without manual intervention.

Data Preprocessing & Cleaning: The system will handle missing values, remove duplicates, and filter irrelevant data to improve data quality and reliability for further analysis.

Feature Engineering & Transformation: The tool will generate meaningful features such as total spend, purchase frequency, and customer segments to enhance analytical capabilities.

Statistical Analysis & Trend Identification: The system will apply statistical techniques like mean, variance, and correlation, along with distribution analysis, to identify sales patterns and customer behaviour.

Scalable Data Storage using MongoDB: The processed data will be stored in a NoSQL database (MongoDB), enabling flexible schema design and efficient handling of large datasets.

Query Processing & Insight Generation: The system will use MongoDB queries and aggregation to extract insights such as top customers, best-selling products, and revenue trends.

Big Data Simulation & Conceptual Understanding: The project will simulate large-scale data processing and demonstrate big data concepts, including an overview of Hadoop workflow for handling massive datasets.

User-Friendly Visualization & Reporting: The system will present results through simple visualizations and reports, making it easier for users to interpret and utilize the insights effectively.

1.3 OVERVIEW

The E-commerce Data Pipeline & Analytics System is developed to efficiently transform raw e-commerce data into meaningful insights. The system collects data related to customers, products, and orders, and processes it through data cleaning and preprocessing techniques to ensure accuracy and consistency. It further enhances the dataset using feature engineering to derive useful metrics such as customer spending and purchase frequency. Statistical analysis is performed to identify patterns, relationships, and sales trends. The processed data is stored in MongoDB for scalable and flexible management. The system also demonstrates basic big data concepts, providing a complete pipeline for data processing, storage, and analysis.

2. SOURCE CODE

Main.py

```
# =====  
# E-COMMERCE DATA PIPELINE & ANALYTICS SYSTEM  
# =====  
  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
import os  
from pymongo import MongoClient  
import kagglehub  
  
print("\n 🚀 Starting E-commerce Data Pipeline...\n")  
  
# -----  
# STEP 1: DATA COLLECTION  
# -----  
print(" 📄 Downloading dataset...")  
  
path = kagglehub.dataset_download("carrie1/ecommerce-data")  
file_path = os.path.join(path, "data.csv")  
  
df = pd.read_csv(file_path, encoding='ISO-8859-1')  
  
print(" ✅ Dataset Loaded")  
print(df.head())  
  
# -----  
# STEP 2: DATA PREPROCESSING  
# -----  
print("\n 🧹 Cleaning Data...")  
  
df.rename(columns={  
    "InvoiceNo": "OrderID",  
    "InvoiceDate": "OrderDate",  
    "UnitPrice": "Price"  
}, inplace=True)
```

```

# Remove missing & duplicates
df.dropna(subset=["CustomerID"], inplace=True)
df.drop_duplicates(inplace=True)

# Convert date
df["OrderDate"] = pd.to_datetime(df["OrderDate"])

print("✅ Preprocessing Done")

# -----
# STEP 3: FEATURE ENGINEERING
# -----
print("\n⚙️ Creating Features...")

# Total purchase amount
df["TotalAmount"] = df["Quantity"] * df["Price"]

# Order frequency per customer
order_freq = df.groupby("CustomerID")["OrderID"].nunique().reset_index()
order_freq.columns = ["CustomerID", "OrderFrequency"]

df = df.merge(order_freq, on="CustomerID")

print("✅ Features Created")

# -----
# STEP 4: STATISTICAL ANALYSIS
# -----
print("\n📊 Statistical Analysis")

print("Mean Sales:", df["TotalAmount"].mean())
print("Variance:", df["TotalAmount"].var())

print("\nCorrelation Matrix:\n")
print(df.corr(numeric_only=True))

# -----
# STEP 5: VISUALIZATION
# -----

```

```

print("\n 📊 Generating Graphs...")

df.set_index("OrderDate", inplace=True)

# Monthly trend (optimized)
sales_trend = df["TotalAmount"].resample("M").sum()

plt.figure()
sales_trend.plot()
plt.title("Monthly Sales Trend")
plt.xlabel("Date")
plt.ylabel("Revenue")
plt.show()

# Distribution (sampled for speed)
sns.histplot(df["TotalAmount"].sample(5000), bins=50)
plt.title("Sales Distribution")
plt.show()

# -----
# STEP 6: MONGODB STORAGE
# -----
print("\n 🗄 Connecting to MongoDB...")

client = MongoClient("mongodb://localhost:27017/")
db = client["ecommerce_db"]
collection = db["orders"]

# Clear old data
collection.delete_many({})

# Insert new data
collection.insert_many(df.reset_index().to_dict("records"))

print(" ✅ Data Stored in MongoDB")

# -----
# STEP 7: MONGODB QUERIES
# -----
print("\n 🔍 Top 5 Customers by Spending:")

```

```

pipeline = [
    {"$group": {"_id": "$CustomerID", "total_spent": {"$sum": "$TotalAmount"}}},
    {"$sort": {"total_spent": -1}},
    {"$limit": 5}
]

for doc in collection.aggregate(pipeline):
    print(doc)

print("\n 🎉 PROJECT COMPLETED SUCCESSFULLY\n")

```

App.py

```

# =====
# E-COMMERCE DASHBOARD + QUERY CONSOLE
# =====

import streamlit as st
import pandas as pd
from pymongo import MongoClient
import matplotlib.pyplot as plt
import json

# -----
# PAGE CONFIG
# -----

st.set_page_config(page_title="E-commerce Dashboard", layout="wide")
st.title(" 🛒 E-commerce Data Analytics Dashboard")

# -----
# LOAD DATA FROM MONGODB
# -----

@st.cache_data
def load_data():
    client = MongoClient("mongodb://localhost:27017/")
    db = client["ecommerce_db"]
    collection = db["orders"]
    data = list(collection.find())
    return pd.DataFrame(data)

```

```

df = load_data()

if df.empty:
    st.error("⚠️ No data found. Run main.py first.")
    st.stop()

# -----
# DATA CLEANING
# -----
df["OrderDate"] = pd.to_datetime(df["OrderDate"])
df["TotalAmount"] = pd.to_numeric(df["TotalAmount"], errors="coerce")
df.dropna(subset=["CustomerID"], inplace=True)

# -----
# SIDEBAR FILTERS
# -----
st.sidebar.title("🔍 Filters")

customer_ids = sorted(df["CustomerID"].unique())
selected_customer = st.sidebar.selectbox("Select Customer ID", customer_ids)

# -----
# KPI METRICS
# -----
st.subheader("💎 Key Metrics")

col1, col2, col3 = st.columns(3)
col1.metric("Total Revenue", f"{df['TotalAmount'].sum():.2f}")
col2.metric("Total Orders", df["OrderID"].nunique())
col3.metric("Total Customers", df["CustomerID"].nunique())

# -----
# SALES TREND
# -----
st.subheader("📊 Monthly Sales Trend")

df.set_index("OrderDate", inplace=True)
sales_trend = df["TotalAmount"].resample("M").sum()

fig, ax = plt.subplots()

```

```

sales_trend.plot(ax=ax)
st.pyplot(fig)

# -----
# TOP CUSTOMERS
# -----
st.subheader("🏆 Top 5 Customers")

client = MongoClient("mongodb://localhost:27017/")
collection = client["ecommerce_db"]["orders"]

pipeline = [
    {"$group": {"_id": "$CustomerID", "total_spent": {"$sum": "$TotalAmount"}}},
    {"$sort": {"total_spent": -1}},
    {"$limit": 5}
]

top_customers = list(collection.aggregate(pipeline))
st.dataframe(pd.DataFrame(top_customers))

# -----
# CUSTOM QUERY SECTION ★
# -----
st.subheader("🧠 Custom MongoDB Query Console")

st.markdown("Enter a MongoDB **aggregation pipeline (JSON format)**")

default_query = """
[
    {"$group": {"_id": "$CustomerID", "total": {"$sum": "$TotalAmount"}}},
    {"$sort": {"total": -1}},
    {"$limit": 5}
]
"""

query_input = st.text_area("Enter Query", default_query, height=200)

if st.button("Run Query"):
    try:
        query = json.loads(query_input)

```

```

result = list(collection.aggregate(query))
result_df = pd.DataFrame(result)

st.success("✅ Query Executed Successfully")
st.dataframe(result_df)

except Exception as e:
    st.error(f"❌ Error: {e}")

# -----
# CUSTOMER DETAILS
# -----
st.subheader(f"👤 Selected Customer: {selected_customer}")

filtered_df = df[df["CustomerID"] == selected_customer]

st.write("Total Orders:", filtered_df["OrderID"].nunique())
st.write("Total Spend:", round(filtered_df["TotalAmount"].sum(), 2))

st.dataframe(filtered_df.head())

# -----
# FOOTER
# -----
st.markdown("---")
st.markdown("🚀 Built with Streamlit + MongoDB + Python")

```

3. OUTPUT

Output:

```
🚀 Starting E-commerce Data Pipeline...

📁 Downloading dataset...
✅ Dataset Loaded
InvoiceNo StockCode Description ... UnitPrice CustomerID Country
0 536365 85123A WHITE HANGING HEART T-LIGHT HOLDER ... 2.55 17850.0 United Kingdom
1 536365 71053 WHITE METAL LANTERN ... 3.39 17850.0 United Kingdom
2 536365 84406B CREAM CUPID HEARTS COAT HANGER ... 2.75 17850.0 United Kingdom
3 536365 84029G KNITTED UNION FLAG HOT WATER BOTTLE ... 3.39 17850.0 United Kingdom
4 536365 84029E RED WOOLLY HOTTIE WHITE HEART. ... 3.39 17850.0 United Kingdom

[5 rows x 8 columns]

🔪 Cleaning Data...
✅ Preprocessing Done

⚙️ Creating Features...
✅ Features Created

📊 Statistical Analysis
Mean Sales: 20.613637872132745
Variance: 185203.03196177285
```

```
Correlation Matrix:

Quantity Price CustomerID TotalAmount OrderFrequency
Quantity 1.000000 -0.001243 -0.003457 0.916130 0.004496
Price -0.001243 1.000000 -0.004524 -0.129311 0.002027
CustomerID -0.003457 -0.004524 1.000000 -0.002130 -0.031221
TotalAmount 0.916130 -0.129311 -0.002130 1.000000 0.005009
OrderFrequency 0.004496 0.002027 -0.031221 0.005009 1.000000
```

```
📁 Connecting to MongoDB...
✅ Data Stored in MongoDB

🔍 Top 5 Customers by Spending:
{'_id': 14646.0, 'total_spent': 279489.02}
{'_id': 18102.0, 'total_spent': 256438.49}
{'_id': 17450.0, 'total_spent': 187322.16999999998}
{'_id': 14911.0, 'total_spent': 132458.73}
{'_id': 12415.0, 'total_spent': 123725.45}

🎉 PROJECT COMPLETED SUCCESSFULLY
```


Dashboard:

>>

Deploy ⋮



E-commerce Data Analytics Dashboard



Key Metrics

Total Revenue

8,278,519.42

Total Orders

22190

Total Customers

4372



Monthly Sales Trend

<<

Deploy ⋮

OrderDate



Top 5 Customers

	_id	total_spent
0	14646	279489.02
1	18102	256438.49
2	17450	187322.17
3	14911	132458.73
4	12415	123725.45

<<

Deploy ⋮



Custom MongoDB Query Console

Enter a MongoDB aggregation pipeline (JSON format)

Enter Query

```
[
  {
    "$group": {
      "_id": "$CustomerID",
      "total": {
        "$sum": "$TotalAmount"
      }
    },
    {"$sort": {"total": -1}},
    {"$limit": 5}
  ]
```

Run Query



Selected Customer: 12346.0

Total Orders: 2

Total Spend: 0.0

OrderDate	_id	OrderID	StockCode	Description	Quantity	Price	CustomerID	Country	TotalAmount	O
2011-01-18 10:01:00	69f1f4e1cde0115f2de1876f	541431	23166	MEDIUM CERAMIC TOP STORAGE JAR	74215	1.04	12346	United Kingdom	77183.6	
2011-01-18 10:17:00	69f1f4e1cde0115f2de18774	C541433	23166	MEDIUM CERAMIC TOP STORAGE JAR	-74215	1.04	12346	United Kingdom	-77183.6	

4. CONCLUSION

The E-commerce Data Pipeline & Analytics System successfully demonstrates how raw and unstructured data can be transformed into meaningful insights through a systematic approach. By implementing data preprocessing techniques, the project ensures data quality and consistency, which is essential for accurate analysis. Feature engineering and statistical methods help in uncovering important patterns such as customer behavior, purchasing trends, and product performance. The use of MongoDB provides a flexible and scalable solution for storing and querying large datasets efficiently. Additionally, the inclusion of big data concepts offers an understanding of how such systems can be extended to handle massive real-world data. Overall, this project highlights the importance of data engineering in modern e-commerce and provides a practical foundation for building intelligent, data-driven decision-making systems.

5. REFERENCES

- [1] **MongoDB, Inc. (2025).** *MongoDB PyMongo Documentation*. MongoDB Docs. A comprehensive reference for the official Python driver (PyMongo), covering installation, connection setup, CRUD operations, aggregation, indexing, and security features for building scalable applications with MongoDB.
- [2] **MongoDB, Inc. (2025).** *MongoDB with Python*. MongoDB Docs. Guides developers through integrating MongoDB with Python frameworks (Flask, FastAPI, Django), using ORMs like MongoEngine and Beanie, and leveraging PyMongo Async for modern data pipelines in production e-commerce systems.
- [3] **DataCamp. (2025).** “MongoDB Aggregation Pipeline Tutorial in Python with PyMongo.” *DataCamp Tutorials*. Explains how to design and implement aggregation pipelines in MongoDB using PyMongo, with examples of \$match, \$group, \$lookup, and \$project stages—essential for processing customer, order, and product data in e-commerce pipelines.